Private Payments on Stellar:
A Client-Proved Multi-Asset Shielded Note Pool with On-Chain Recovery

StellarKey
support@stellarkey.io

2026-11-06

StellarKey 1.1.0 · Protocol V2.1
Development Testnet implementation; not for real value

Abstract

Coupling shielded-pool actions to growth of a finite commitment tree can turn tree exhaustion into an inability to withdraw previously deposited value. We describe StellarKey Private Balance, a browser-proved multi-asset note pool on Stellar, and an implemented separation between commitment growth and full-input withdrawal. Protocol V2 composes private 17-level and 47-level ternary paths under one global root, binds nullifiers to exact global positions, and adds a proof-bound exit that consumes one or two inputs in full without appending commitments. Protocol V2.1 removes the published action kind: every action is one uniform shape whose public settlement legs (public inflow, public outflow, and a proof-bound appends flag) classify it, so no separate kind label is published. Ordinary wallet withdrawals append commitments even when no private change remains. Non-appending capacity exits still reveal that the selected inputs left no change; this does not reveal whether the wallet owns other notes. Internal transfers authorize value movement without publishing the selected notes, recipient, amount or asset; deposits and withdrawals retain their public endpoints and amounts.

The Circom 2.2.3 implementation has 40,647 optimized constraints and 12 public signals. A fresh development Testnet deployment supports XLM and USDC. Regression checks exercise saturation, positions beyond 64 bits, shared replay protection, transaction rollback, independent archive and leaf cursors, and balanced but invalid exit witnesses. We give conditional state-transition arguments, describe an authenticated recovery transcript and the browser's durable proof-exposure rules, and distinguish synthetic validation from dated Testnet evidence. The contribution is a systems construction and implementation; it is neither a new proving system nor a claim of unbounded storage. Groth16/BN254, a single-party development setup, public transaction sources, metadata leakage, token backing, and ledger availability remain explicit limitations. The implementation is not approved for real value or Mainnet.

Implementation baseline: `v1.1.0`. Planned release tag; this working-tree candidate is not yet published. Artifact identities bind the protocol described here; source links become immutable after publication. Dated deployment, benchmark, and test evidence retains the scope and limitations in Sections 16–17.

Contents

1 Design goals and non-goals **3**
- 1.1 Contributions and scope . . . 3
- 1.2 Adversarial model and assumptions . . . 4

2 System architecture **4**

3 Deployment-bound keys and private addresses **5**

4 Cryptographic suite **6**

5 Notes, commitments, and output packages **7**

6 Fixed-shape actions **7**

7 Groth16 statement **8**

8 Ternary incremental Merkle tree **9**
- 8.1 State and transition relation . . . 10
- 8.2 Multi-note withdrawal . . . 10

9 Pool contract state transition **11**

10 Recipient and sender recovery **12**

11 Canonical archive and restoration **13**
- 11.1 Archive-record restoration . . . 13
- 11.2 Shared contract liveness and read-only simulation . . . 13

12 RPC authenticity and client finality **14**

13 Direct submission and held-balance recovery **15**
- 13.1 Review, fees, and confirmation . . . 15
- 13.2 Held-balance self-recovery . . . 16
- 13.3 Current-format state . . . 16

14 Local state and execution boundary **16**

15 Privacy boundary **17**
- 15.1 Conditional security arguments . . . 18

16 Evaluation and reproducibility **19**
- 16.1 Validation method . . . 20
- 16.2 Integrated browser proving experiment . . . 21
- 16.3 Contemporary comparison . . . 21
- 16.4 Artifact identity and reproduction . . . 22

17 Deployment and trust status **22**

18 Normative sources and operational guidance **23**

References **25**

1 Design goals and non-goals

Protocol V2.1 is the release name. Its encoded compatibility identifier remains `3` (u16 `0x0003`) in manifests, key derivation, note/context encodings, and HPKE info. The name is not a decimal value to serialize; naming it V2.1 does not change the existing keys, proofs, or deployed contract. Application releases use a separate version sequence.

The implementation targets five properties:

1. **Value conservation.** A proof and the pool contract jointly enforce exact 63-bit integer balance across private inputs, three private outputs, and public value. The fixed three-output protocol format remains unchanged.
2. **Private internal transfers.** A transfer does not publish its private amount, recipient, selected notes, or change role.
3. **Seed-based recovery.** Seed plus authenticated chain data can recover owned notes and spent status without a StellarKey server. Outgoing recipient fingerprints and memos also recover when the sender included recovery records.
4. **Fail-closed clients.** A wallet may spend only after its archive transcript, action count, incremental Merkle state, and contract head agree. Independent RPC corroboration is mandatory for initial or full-history synchronization and enabled by default for routine synchronization.
5. **Deployment and asset binding.** Every key hierarchy, address, artifact set, and local record is bound to a network, realm, and contract. Every note commits to its full asset field, which the append-only registry permanently maps to one index.

The design does not hide network metadata, the pool, action timing, the transaction source, deposits, or withdrawals. Protocol V2.1 no longer publishes an action kind; boundary actions remain identifiable through their public token movements, which the kind label only duplicated. Deposits and withdrawals reveal their asset; an internal transfer does not publish it. Direct mode self-submits from the user's public Stellar account and is now the only supported submission path. It does not guarantee a minimum privacy set, protect a compromised browser, or make an unaudited single-party proving setup safe for real value.

1.1 Contributions and scope

This paper addresses a specific liveness failure: can exhaustion of commitment slots alone prevent redemption of an otherwise valid, owned note? The answer in the implemented V2 transition is no, provided the holder withdraws the selected input value in full. This is narrower than unconditional withdrawal availability. An arbitrary partial amount may require change and therefore free commitment slots. Ledger resource limits, proof availability, transaction fees, token-transfer semantics and canonical-state availability remain preconditions.

The contributions are (i) a growth-independent exit in the same nullifier domain as normal spends; (ii) an exact-position implementation across a 64-level circuit, Soroban u128 values, browser BigInt values and durable decimal encodings; and (iii) recovery and multi-note withdrawal rules that preserve authenticated history and prior proof exposure across interruption. The 17+47 composition is algebraically one fixed-depth tree, not a recursive proof, dynamic accumulator, or novel Merkle primitive. We do not claim priority over all shielded-pool exit constructions. Sections 7–9 specify the construction; §15 states conditional security arguments; §16 gives reproducible evidence and contemporary comparisons.

1.2 Adversarial model and assumptions

An adversary may choose arbitrary action data and ciphertexts, submit concurrent transactions, copy exposed proofs, control recipients, and return stale or malformed RPC data. It may control one RPC endpoint, the network transport, or the application origin in separate threat cases. A malicious application origin that changes the verifier or client before unlock is outside the honest-client privacy assumption; hashes delivered by that same compromised origin are not an independent trust root. Compromise of the unlocked browser, seed or relevant viewing keys defeats the corresponding confidentiality guarantees.

The conditional arguments assume an honestly generated Groth16 reference string with erased Phase 2 trapdoor, knowledge soundness and zero knowledge for the chosen relation, binding/ preimage resistance of the instantiated hashes, appropriate PRF/KDF and AEAD security, secure randomness, and correct execution of Stellar consensus and Soroban's atomic rollback semantics. The deployed single-party setup does not establish the erased-trapdoor assumption. Agreement of two RPC providers is corroboration, not a locally verified consensus proof; colluding providers can defeat that trust boundary. Asset redeemability further assumes honest exact-transfer token behavior and sufficient accessible custody.

2 System architecture

There is no application backend holding private note state or proving on the user's behalf. Browsers use Stellar RPC for public ledger data and submission.

One deployment is one privacy and state domain. Its constructor binds the network ID, realm, initial asset administrator, guardian, circuit hash, verification-key hash, Poseidon2 parameter hash, tree parameters, address format, encryption suite, and deployment-binding hash. The contract maintains one shared tree and an append-only asset registry. The administrator can add assets or set them `Active`/`ExitOnly`; entries cannot be deleted or reindexed.

Component	Implemented responsibility
Browser vault	Holds the Stellar seed and derives deployment-bound private keys only after explicit user intent.
Isolated worker	Scans ciphertext, maintains private note state, constructs witnesses, and proves actions.
Public Merkle cache	Stores authenticated, disposable commitment nodes and checkpoints; it contains no note plaintext.
Pool contract	Verifies proofs, enforces nullifier uniqueness, updates the shared tree and registry, moves boundary assets, and appends recovery records.
Primary Stellar RPC	Supplies ledger state, simulation, submission, confirmation, and archive restoration.
Different-origin witness RPC	Corroborates network identity, any retained deployment-checkpoint ledger, a current overlapping ledger hash, and the complete contract head during mandatory initial/full-history checks and, by default, routine checks.
Static application origin	Serves hash-pinned manifests, circuit Wasm, proving material, and verification data.

Transfer and withdrawal need no user Soroban authorization: the proof authorizes the state transition. Direct mode still signs a transaction from the user's public Stellar account, so that source

links the action to the account. Private action forms default network fees to that account and can select another software account held in the same wallet. This changes only the public XLM fee payer, not the deposit source, private identity, selected notes, or recipient. Watch-only and unsupported hardware accounts are listed but cannot be selected for private fee signing. Both the inner source and alternate fee payer remain public; fee sponsorship does not hide either identity.

The unsigned review rejects fee-bump envelopes supplied by another party. For an explicitly selected payer, the wallet first signs the exact reviewed inner transaction, then locally constructs and signs a constrained fee-bump envelope. The payer is bound before proof disclosure and retained in the review, encrypted journal, and any multi-step approval. Its spendable public XLM is checked after reserves and selling liabilities. The resource fee is charged once; the outer envelope adds one inclusion-fee operation. Neither per-step nor cumulative approval limits are raised automatically. Deleting or changing the payer fails closed without falling back to charging the current account.

Because the inner signature affects the outer hash, signing atomically journals the submitted outer hash together with the reviewed inner hash before broadcast. Polling, explorer links and resume use the actual submitted envelope and its retained payer and fee limits.

Reviews use the selected wallet account as the inner source. Direct submission is the only supported route. No public relayer address or fee exists in the contract action or archive, and there is no helper-earnings capability.

3 Deployment-bound keys and private addresses

A supported software account derives a 64-byte privacy session root from the raw Stellar seed plus the protocol version, network ID, realm ID, pool contract ID, and Stellar account public key. Domain-separated expansion produces:

- `ask`, the private spending authorization field;
- `nk`, the nullifier derivation field;
- an incoming X25519 viewing key;
- an outgoing viewing key;
- a base owner commitment; and
- a separate deployment-bound local-storage key.

A four-byte diversifier derives an address-specific owner commitment and an address-specific X25519 key pair. All three outputs in one action deliberately carry one common clear action diversifier. For a transfer this is the recipient address diversifier; change and dummy lanes are constructed to match it. Deposits and withdrawals choose a fresh random action diversifier. This prevents the clear bytes from identifying output roles inside an action while retaining the reviewed X25519 scan path. It does not hide the diversifier: reusing a receive address can still link the whole actions that carry it. Spending authority stays common to the account while issued receive addresses can rotate.

First setup generates a random non-zero receive diversifier. Session initialization preserves an existing supported receive address exactly and rejects an unsupported default identity. Each new receive opening creates a fresh address after the user explicitly selects Private. Focusing a tab alone is not issuance intent. Switching tabs or assets within the same pool preserves the request; New address explicitly replaces it. If issuance is unavailable, the wallet offers warned, explicit reuse of the saved address rather than silently sharing it. The provider records each issued diversifier in

encrypted local state before publishing the address; receipts from current-format issued addresses remain recoverable. Rotation does not provide forward secrecy against compromise of the seed or viewing key.

Issuance records up to 65,536 diversifiers and refuses to reissue a recorded value. Rotation excludes the zero diversifier and current address. Full verification, failed-verification rollback, and encrypted backup restore preserve the issued-address history. A recorded collision fails safely and asks for a fresh attempt; reaching the local bound stops issuance without disabling existing addresses. This is local reuse prevention, not hidden-diversifier cryptography or a guarantee across independent devices, old lost history, seed-only recovery, or deliberate local-data removal.

In the current Base58 address format, `tskpay_` addresses are exactly 128 ASCII characters on Testnet and `skpay_` addresses are exactly 127 on Mainnet. The decoded form contains a one-byte format marker, an 84-byte payload, and a four-byte checksum. The payload contains:

- a 16-byte deployment tag;
- a four-byte diversifier;
- a 32-byte owner commitment; and
- a 32-byte recipient X25519 public key.

The prefix is included in the checksum, and the deployment tag is derived from the immutable deployment binding. Wrong-network, wrong-deployment, malformed, and aliased-key addresses fail before payment construction. The four-byte checksum makes accidental corruption or a mistyped address overwhelmingly likely to fail, but a 32-bit checksum is an error detector, not a guarantee or an authentication code. Address reuse can still correlate the same recipient off-chain.

4 Cryptographic suite

New recipient envelopes use the V2.1 HPKE info context. Recovery also reads early V2.1 development envelopes that used HPKE info version 2, subject to authenticated V2.1 plaintext, the deployment context and full commitment. Obsolete V2 note plaintexts remain rejected; this is not migration from the retired V2 pool.

The implemented V2.1 suite is:

Purpose	Construction
Proof system	Groth16 over the BN254 scalar field
Circuit hash	Poseidon2 over BN254 $\mathbb{F}_r$, width 4, rate 3, capacity 1, x^5, 8 full rounds and 56 partial rounds
Recipient encryption	RFC 9180 base mode: DHKEM(X25519, HKDF-SHA256), HKDF-SHA256, AES-128-GCM
Outgoing recovery	Outgoing-viewing-key HKDF-SHA256 derivation and AES-128-GCM, with deployment- and output-bound associated data
Archive transcript	SHA-256 hash chain rooted in a deployment-specific genesis value
Address body	Base58 with a prefix-bound SHA-256 checksum and deployment tag

The primitive references are Groth's pairing-based argument, Poseidon2, and HPKE, RFC 9180. Stellar's CAP-0074 specifies the native BN254 operations and explicitly warns that BN254 no longer provides 128-bit security. AES-128-GCM and the parameter sizes elsewhere in this suite therefore do not establish a uniform 128-bit security claim for the complete protocol. Security also depends

on the exact circuit, hash parameters, setup, randomness, encodings, and their composition; citing these primitives does not constitute a proof or independent review of this implementation.

Scalar-field elements used by the circuit and public signals are canonical 32-byte big-endian values below $\mathbb{F}_r$; BN254 proof coordinates instead use the curve's base field $\mathbb{F}_q$. Protocol integers use fixed-width encodings and private values are bounded to 63 bits. The manifest directly pins the runtime constants and artifact hashes it exposes; those hashes transitively bind the remaining circuit, contract, hash, and encoding parameters in the authenticated artifacts.

5 Notes, commitments, and output packages

A 128-byte decrypted note plaintext contains the protocol version, a real/dummy flag, value, diversifier, owner commitment, note randomness ρ, an optional memo of at most 32 bytes, the immutable four-byte asset-registry index, and 11 reserved zero bytes.

The note commitment binds the deployment context field, asset field, owner commitment, value, and ρ. The owner commitment already incorporates the address diversifier. The memo is authenticated by encryption but is not a separate circuit input or note-commitment field.

Every action publishes exactly three non-zero, pairwise-distinct commitments. FullInputExit retains these fixed public fields but inserts none into the tree. Zero-value dummy notes represent unused outputs; they are never absent or zero commitments. Each receives fresh randomness, owner/key material, recipient ciphertext, outgoing ciphertext, and a normal-looking commitment. Dummy lanes share the same clear action diversifier as the real lanes, as described in §3; they do not carry independent clear diversifiers that would reveal their role. The circuit derives `outputReal` from `value != 0`; the wallet does not provide output role as independent witness data.

Each 370-byte output package consists of:

- a 32-byte commitment;
- a 181-byte recipient envelope; and
- a 157-byte outgoing envelope.

The wallet independently randomizes real/dummy input lane ordering and recipient/change/dummy output lane ordering before building commitments and encryption envelopes.

6 Fixed-shape actions

Every action has two input lanes and three output lanes. Real inputs carry a note witness and a depth-64 membership path. Dummy inputs carry a fresh secret that produces a non-zero public dummy nullifier, while their unused membership data has one canonical private zero form. The public slots do not disclose the private `inputReal` selectors.

Protocol V2.1 publishes no action kind. Two proof-bound settlement legs classify every action, and they add nothing the ledger does not already show: a settlement inward is a deposit, a settlement outward is a withdrawal, and an action with both legs quiet is an internal transfer. A proof-bound `appends` flag selects whether the three commitments enter the tree; the circuit requires it to be one unless the outflow leg is active, and a non-appending action proves all three outputs dummy. Wallet policy sets it to one in normal operation, including exact-input withdrawals. Only a fresh tree count matched to the authenticated checkpoint can select a non-appending capacity exit, when

fewer than three commitment slots remain. Such an exit remains publicly distinguishable through the flag and unchanged tree. The contract permits non-appending exact-input withdrawals before saturation too; this restriction is wallet policy, not a consensus privacy guarantee.

Settlement	Public information	Private statement
Inflow only (deposit)	Source, amount, pool, registry index, asset, three commitments, nullifiers, timing and fee payer	Both inputs are dummy; at least one output is real, and private outputs sum to the deposited value.
Both legs quiet (transfer)	Pool, anchor root, three commitments, nullifiers, timing and transaction source	At least one owned input and one real output; asset, recipient, amount, note count and output roles remain private.
Outflow only (withdraw)	Public recipient and amount, pool, registry index, asset, anchor root, three commitments, nullifiers, timing and transaction source	At least one owned input; any remainder occupies private outputs.
Outflow with `appends = 0` (full-input exit)	Ordinary withdrawal boundary information plus the public absence of private change from the selected inputs	Selected real inputs equal the public withdrawal; all three output values are zero, and their bound dummy commitments are not inserted.

For all actions the circuit enforces the same kind-free balance equation:

$$\sum v_{\text{in}} + v_{\text{inflow}} = \sum v_{\text{out}} + v_{\text{outflow}}.$$

Each individual value and one post-equality total receive a 63-bit range check.

7 Groth16 statement

The production-shaped development circuit has 40,647 constraints, 12 public inputs, and 410 private inputs when compiled with Circom 2.2.3 and `--O2`. Its setup uses the pinned power-17 Phase 1 transcript. The R1CS domain selected by the setup is 65,536; the transcript has sufficient degree for that domain.

The 12 public inputs, in verifier order, are:

1. deployment context field;
2. boundary asset field (zero for an internal transfer);
3. public settlement inflow;
4. public settlement outflow;
5. appends flag;
6. anchor root;
7. canonical action field;
8. first nullifier;
9. second nullifier;
10. first output commitment;
11. second output commitment; and
12. third output commitment.

The private witness includes the full action asset field and proves owner authorization, real-input membership, correct real or dummy nullifier derivation, distinct spent leaves, three output commitments, fixed action shape, and conservation of value. The conservation equation is kind-free: at most one settlement leg carries value, and both legs quiet is an internal transfer. Deposits and withdrawals prove that the private full asset field equals the public registered boundary asset; quiet transfers require the public boundary field to be zero. The canonical action field hashes the network, realm, pool, optional boundary asset/index, nonce, anchor root, complete outputs, and public endpoints. It contains no relayer metadata. A non-zero circuit constraint gives this public field a non-zero Groth16 input coefficient; a proof-mutation regression confirms that changing it invalidates the proof. The contract derives the field itself.

Encryption is intentionally outside the circuit. A valid proof binds an output commitment but does not prove that its ciphertext is decryptable or honestly addressed. A malicious sender can therefore burn its own value into an undecryptable output. Wallet-generated change is derived and self-checked before submission, but the protocol cannot repair deliberately malformed ciphertext.

8 Ternary incremental Merkle tree

The commitment tree is ternary and depth-64: a private depth-17 inner path feeds a private depth-47 outer path. Its exact capacity is `3^64 = 3,433,683,820,292,512,484,657,849,089,281` leaves. This is finite. The construction hides both the inner root and outer position under the one public anchor. It exposes no source-subtree selector. The position is $\ell = \ell_{\text{inner}} + 3^{17}\ell_{\text{outer}}$, with each path position constrained to a ternary digit; hence $0 \leq \ell < 3^{64}$.

Actions with the appends flag set append three commitments. A non-appending action — the proof-bound full-input exit — appends none, independently of the remaining slots, and its proof proves all output values zero. Because the flag is one for every ordinary action, a settled spend cannot be told apart from a change-producing withdrawal by its append behaviour; the zero append appears only at tree saturation, where normal actions still return `TreeFull` but a valid exit can consume existing notes against the current root. There is no reset, wraparound, public subtree rollover or reuse of commitment positions. The earlier 43,046,721-action lifetime cliff is removed; the enlarged tree does not make ordinary growth infinite or guarantee an arbitrary partial withdrawal at saturation.

The contract uses u128 positions and archive action counters. The browser uses BigInt, including Merkle keys, nullifiers, scanner cursors, RPC arguments and cross-tab updates, and stores canonical decimal strings. No global position is converted through a JavaScript Number. Bounded local batch lengths, registry indices and ledger sequence numbers remain ordinary bounded integers.

A parent is the three-input Poseidon2 hash over ordered left, middle, and right children. It applies one width-four permutation with those children in the three rate cells and `3 * 2^64` in the capacity cell, returning the first state element. It is not a three-wide permutation. All three rate elements are occupied, so the Merkle parent has no string-domain field. The pinned Poseidon2 length IV separates hashes by arity only. Every other arity-three protocol hash puts a distinct domain constant in rate slot zero; separation from the raw parent additionally relies on Poseidon2 preimage and collision resistance. Arity, ordering, depth, empty roots, the length IV, and the domain-slot convention are consensus parameters.

The contract and browser use the same generated empty roots and canonical hash implementation. The contract maintains a 128-node frontier—two slots for each of 64 levels—and recomputes the current root after an appending action. A full-input exit preserves both the root and frontier.

The browser persists an authenticated incremental Merkle node store. Its checkpoint binds the deployment, archive cursor, transcript head, commitment count, root, and complete frontier. Verified batches atomically append nodes; non-contiguous or corrupted cache state fails closed. An ordinary spend reads the selected logarithmic paths instead of reconstructing every historical commitment. The node cache is public, disposable, and excluded from backups.

Non-deposit actions may spend against a known recent root. The root window is 1,440 ledgers. A spend against the current root refreshes it transactionally, and `touch_root` permissionlessly keeps the current canonical root available without moving value.

8.1 State and transition relation

Write the logical state as $S = (T, n, R, N, A, h, G)$: the commitment tree and frontier T, next leaf index n, retained roots R, spent nullifiers N, next archive action index A, transcript head h, and immutable-index asset registry G. Ledger timestamps, TTL metadata and token custody are additional execution state. A successful action changes the progress counters as follows:

Accepted action	Counter and tree update
Appends = 1 (deposits, transfers, withdrawals with change)	$A' = A + 1$, $n' = n + 3$; append all three commitments, requiring $n + 3 \leq 3^{64}$.
Appends = 0 (full-input exit)	$A' = A + 1$, $n' = n$ and $T' = T$; no commitment capacity guard.

For every accepted record r_A, the record's starting leaf index is the pre-transition n, and $h' = \mathrm{SHA256}(\mathrm{Encode}(r_A, h))$. The domain- and deployment-bound genesis initializes h. Sequential scanning checks the expected archive index separately from the expected leaf position; its warm checkpoint binds both. Thus n is not inferred as $3A$. In reachable states $0 \leq n \leq 3A$ and n is divisible by three, but those arithmetic conditions alone do not authenticate a history.

The archive's u128 counter is also finite. Its checked increment rejects overflow; a mathematical exhaustion of all 2^{128} record indices is outside the exit claim. Even before that bound, storage rent, data availability and recovery time are practical constraints. “Capacity-independent” in this paper refers only to commitment-tree capacity, not to every resource of the system.

8.2 Multi-note withdrawal

A proof accepts at most two real inputs and its total is below 2^{63}. The wallet first prefers an exact single note or pair. If more inputs are needed, it reviews disjoint groups of one or two available notes, each within the circuit amount range. A full-balance withdrawal consists entirely of FullInputExit steps; a partial request can have a final ordinary withdrawal that creates change. The wallet never increases the requested public amount to make an exit fit.

One approval covers at most 64 steps and 15 minutes, with per-step and cumulative network-fee limits. Each review is bound to its asset, public recipient, input identifiers, exact amount, change and entrypoint. Before preparing the next step, the preceding step must be canonically confirmed; acceptance or timeout is not confirmation. This is a sequence of individually atomic withdrawals, not one atomic aggregate withdrawal. An interrupted sequence may have completed a prefix. A new approval is required for further work beyond its bounds. Aggregate amounts may use the

token's signed-128-bit range, while every proof retains the 63-bit bound. No consolidation that creates leaves is required for a full-balance exit.

9 Pool contract state transition

The pool constructor rejects any network or artifact configuration that differs from the compiled constants and recomputed deployment binding. The implemented contract exposes one uniform shielded-action entrypoint whose settlement legs classify deposits, transfers, withdrawals and full-input exits, read-only configuration/head methods, permissionless current-root refresh, guardian-authorized deposit pause, and an administrator-authorized append-only asset registry.

Registry indices are permanent and contiguous. The asset administrator can add a contract address, switch it between `Active` and `ExitOnly`, and hand authority to a new administrator through a propose/accept sequence. `ExitOnly` blocks deposits but preserves transfers and withdrawals for existing notes. This means the registry status does not prohibit those actions; proof, root, capacity, asset, and runtime conditions still apply. The contract cannot delete, replace, or reindex an entry.

Admission checks administrator authorization, contract-address encoding, uniqueness, and index capacity; it does not verify a Stellar Asset Contract executable or attest token behavior. Boundary transfers expect a compatible token interface. The curated XLM and USDC entries are Stellar Asset Contracts, but admission alone is not proof of backing or withdrawal availability. Issued assets retain their applicable authorization, trustline, and clawback rules; shielded notes do not remove those underlying asset risks. See the Stellar Asset Contract semantics.

Value conservation is a statement about note accounting, not an independent solvency or redeemability guarantee. The pool calls the admitted token's transfer interface without measuring before/after custody balances. Correct backing therefore assumes honest, exact token-transfer semantics. Where the underlying SAC balance permits it, asset-admin clawback can remove pooled backing and authorization revocation can prevent token movement; neither operation cancels corresponding private notes. Those powers belong to the underlying token administrator, distinct from the pool's registry administrator. A failed withdrawal rolls back its pool state, but cannot repair missing backing or revoked token authorization. The implementation has no automatic insolvency resolution. These are conditional asset risks, not evidence that the advertised Testnet assets have suffered them.

For an action, the contract atomically:

1. resolves and checks the public registered asset when a settlement leg carries value, or requires no public asset for a quiet transfer, and checks canonical encoding;
2. enforces the deposit pause for settlement inflows or verifies a recent anchor root;
3. rejects already-spent durable nullifiers;
4. checks remaining tree capacity when the proof-bound appends flag is set;
5. reconstructs the 12 public signals and verifies the Groth16 proof;
6. persists replay protection;
7. appends all three commitments and updates the root/frontier when appends is set, or preserves them for a non-appending full-input exit;
8. appends the authenticated archive record and transcript head;
9. moves the registered asset only along the active settlement leg; and
10. emits the shielded-action event.

Any failure, including token movement or budget failure, rolls back the whole invocation. Every action persists both public nullifiers. Dummy lanes derive fresh non-zero values, so deposit, transfer and withdrawal replay protection is identical and nothing is distinguishable by its replay treatment.

The guardian can pause all new deposits but cannot rewrite history or bypass proof verification. The separate asset administrator controls admission and status but cannot spend notes or alter historical registry identities. Transfers and withdrawals remain governed by their normal proof, root, and nullifier rules.

10 Recipient and sender recovery

The recipient envelope lets the incoming viewing key test and decrypt an owned note. Its HPKE information and associated data directly bind the protocol context (network, realm, and pool), commitment, action nonce, and output index; the recomputed commitment binds the recovered registered asset and note fields. The outgoing envelope's associated data additionally binds the deployment-binding hash and asset field directly. Wrong-context and tampered ciphertext fail authentication.

Recipient HPKE uses base mode, which does not authenticate a sender identity. Successful decryption and commitment validation establish a recoverable note, not the truth of a memo or the identity of its author. Outgoing-key authentication likewise proves possession of that symmetric key, not a public signature. As specified in RFC 9180, recipient-key compromise can expose past recipient ciphertexts. Address rotation does not provide forward secrecy against compromise of the seed or incoming key from which those address keys are derived.

By default, the outgoing envelope is authenticated by the outgoing viewing key. During a seed-only scan it lets the sender recover an external recipient fingerprint and memo, even when prior local note history is absent. Seed-recovered activity retains the fingerprint, not the full reusable address. Ordinary sends separately retain the full private address in the encrypted recent-recipient list for convenience. Change and dummy outputs use the same envelope sizes and are not presented as external payments.

Advanced privacy offers an account- and deployment-scoped **Recover outgoing payment details** preference. Recovery stays enabled by default. Disabling it requires a separate warning and confirmation while the runtime is idle. The selected mode is encrypted and snapshotted before each action is built; it cannot rewrite an already prepared proof. In minimized mode all three outgoing lanes contain independent random 157-byte fillers instead of sender-recoverable envelopes. Incoming envelopes and the action's fixed shape are unchanged.

Minimized actions do not add recent recipients or copy pending outgoing fingerprints, memos or transaction hashes into permanent activity. Pending safety journals still need payment details until canonical reconciliation. Seed scans continue to recover owned notes, spent status and incoming information; a canonical outgoing amount describes the net private-balance debit. Self-payments may still be recoverable as incoming payments. This is not deletion or forward secrecy: recipients retain their own information, and current-format outgoing records and backups remain readable. The scanner tries recovery-enabled records even when the current preference is minimized. Full encrypted backups and verification rebuilds preserve the setting; a seed-only restore defaults to recovery enabled.

Native WebCrypto imports raw X25519 private material through PKCS#8 and avoids a redundant JavaScript base-point multiplication. The portable implementation remains as a fallback, and both

paths must derive byte-identical shared secrets. Locally owned shared-secret, plaintext, ephemeral-key, and derived-key buffers are cleared in `finally` paths where the runtime permits.

11 Canonical archive and restoration

Every accepted action writes one immutable persistent archive record keyed by its action index. It includes the action/ledger indices, starting leaf index, kind, nonce, anchor and resulting roots, two nullifiers, three complete output packages, and public value and endpoint. Deposit and withdrawal records include the public registry index and asset; transfer records contain neither. No record contains a relayer, fee recipient, or public peer-fee amount. Each record hash commits to the prior transcript head, so deletion, reordering, insertion, or mutation breaks recovery.

A seed-only recovery starts at the deployment genesis, reads records in order, recomputes each record hash and action field, rebuilds the incremental tree, trial-decrypts transfer envelopes against the corroborated registry, validates the full commitment, derives owned nullifiers, and reconciles the final action count, transcript head, frontier, and Merkle root. Only then can the client accept the recovered note state for spending. Recovery does not remove tree-capacity limits, token restrictions, or public fee and restoration requirements.

The corroborated registry and commitment-bound asset field are authoritative for an owned recipient note. A wrong redundant asset index inside an otherwise valid recipient plaintext does not discard that note: the scanner uses the matching registry entry's index and contract identity. This narrow recovery rule does not forgive canonical archive, registry, transcript, or tree corruption, which still fails closed. Neither a recipient-metadata failure nor a canonical validation failure releases an exposed proof's reserved inputs.

11.1 Archive-record restoration

Persistent archive records can move into Stellar state archival after their TTL expires. Restoration discovers only the contiguous unavailable prefix, derives the exact ledger keys locally, freshly simulates candidate restore footprints, and selects the largest safe contiguous batch accepted by fresh simulation and whose simulated resource fee fits an 80%-of-configured-cap budget. Each bounded restore operation is signed and confirmed against its exact transaction hash. A confirmed batch is durable on-chain and advances only in-memory restoration progress. After the selected range finishes, the canonical sync rereads the records, validates the transcript and tree, and persists an encrypted scan checkpoint. An interruption before that sync commits causes a rescan; cancellation or ambiguous status never reports a batch as confirmed.

Restoration therefore reduces round trips and signatures without trusting an RPC-expanded footprint. It is a liveness and fee dependency: confidentiality and ownership remain, but a user who cannot fund restoration may be unable to recover old history at that time.

11.2 Shared contract liveness and read-only simulation

The pool's executable code and instance storage have their own TTLs, separate from archive-record TTLs. Configuration, tree/frontier, archive head, asset count, administrator state, and deposit pause live in instance storage. The contract extends persistent archive and registry entries when writing them, but does not extend its own code/instance TTL. `touch_root` extends only

the temporary known-root entry, not the contract's lifetime. Shared contract restoration or TTL extension therefore needs separately funded maintenance; there is no automatic keepalive service or in-wallet shared-code/instance restoration workflow. Stellar documents these distinct operations in its state archival guides.

For public getters, simulation can temporarily restore archived entries without committing them on-chain. The client accepts this as read-only only when the bounded write footprint consists entirely of exact persistent/code restoration keys, with no authorization and no other state changes. Each restored value must match fresh archived ledger data byte-for-byte, including an expired TTL and a ledger observation no older than the simulation. Unverifiable responses fail closed. This path does not sign, submit, pay for restoration, or retain restoration evidence as a reusable freshness grant. A readable getter is not proof that a subsequent payment fits the wallet's fee cap.

12 RPC authenticity and client finality

The wallet uses a primary RPC selected by the user and a different-origin RPC pinned by the deployment manifest. The shipped default pairs SDF's `https://soroban-testnet.stellar.org` primary with Ankr's `https://rpc.ankr.com/stellar_testnet_soroban` witness. The runtime reuses each loaded contract specification to bound the public read burst, then verifies different URL origins, network and ledger agreement; it cannot prove operator independence if a user selects a custom primary under common control with the witness. The browser-CORS and five-pass state-corroboration screening behind this choice is preserved in `rpc-witness-validation.json`. A second-provider witness check is mandatory when there is no authenticated checkpoint, during seed recovery, and for an explicit full-history check. Routine witness checks are enabled by default but can be disabled in protocol settings. Seed recovery and a full history check always require the witness even when routine checks are disabled. When the witness is used, the wallet compares:

- both network passphrases;
- the manifest-pinned deployment checkpoint hash against each provider that still retains that ledger;
- an overlapping ledger hash at the latest sequence common to both providers;
- the complete pool configuration;
- action count and transcript head; and
- next leaf index, 128-node frontier, and current root.

The two contract heads may be retried a bounded number of times to allow normal ledger skew. If the deployment checkpoint has aged out of either provider's rolling retention window, the authenticated manifest remains the pinned trust anchor; if it has aged out of both, the wallet still requires agreement on a current overlapping ledger and the complete contract head. Any retained copy must match the pinned hash. During a witnessed pass, an unavailable witness, mismatched overlapping ledger hash, stale head, inconsistent tree count, or persistent head disagreement disables spending and preserves the last authenticated snapshot. It is never converted into a false zero balance. If the user disables routine witnessing after a checkpoint exists, later routine passes trust the selected primary RPC's head alone; this weakens protection against a fabricated ledger view but does not weaken the mandatory initial, seed-recovery, or full-history checks.

Using different-origin RPC providers reduces reliance on one fabricated ledger view when their operators are actually independent, but exposes the wallet's pool, timing, ledger ranges, and recovery access pattern to more endpoints. It is an explicit authenticity/privacy tradeoff and an

operator-diversity assumption, not a network-identity-hiding mechanism.

RPC `PENDING`, Horizon acceptance, and timeouts are not ledger confirmation. The UI and durable action journal distinguish preparation, signing/submission, broadcast or ambiguous pending state, canonical confirmation, rejection or failure, and runtime status-unknown. Ambiguous submissions keep their input notes reserved. For an exposed spend proof, absence alone is never sufficient to release them; the separate deposit-envelope recovery rules are described below.

Prepared actions persist an explicit direct submission route, proof-exposure marker and outgoing-history policy in encrypted state. Missing or unsupported fields are rejected before signing, broadcasting or recovery. Restart only rebroadcasts current-format direct signed actions. A disclosed spend proof is reusable in a fresh transaction: envelope failure or maximum-time expiry does not revoke it, and a current anchor can be refreshed. The client durably records exposure before proof-bearing RPC preparation and keeps those inputs reserved on cancellation, rejection, timeout or envelope expiry. Canonical inclusion or observed consumed nullifiers resolve the notes. Unsigned exposed preparations are visibly status-unknown; they are not silently discarded as unsubmitted drafts.

This conservative client does not yet automatically prove a non-current anchor permanently unusable, so an absent exposed spend can remain reserved indefinitely. The old recorded anchor expiry is not sufficient. A proof-bound execution deadline requires a contract/circuit migration. Seed-only recovery cannot reconstruct an unconfirmed proof shared before local journals were lost; absence from the chain is not proof that no such authorization exists. Envelope-based expiry remains relevant to non-reusable deposit authorization; conservative common-ledger corroboration and exact-envelope validation still apply there. A timeout, RPC acceptance, or removed pending record alone is not a success receipt.

13 Direct submission and held-balance recovery

13.1 Review, fees, and confirmation

Private transfers, withdrawals, and consolidation submit only from the user's own Stellar account through the selected RPC. The review explains that the submitting account and public network fee remain visible. The application has no peer discovery, helper approval, helper fee notes or Earn controls. Only current direct-payment drafts and reviews are accepted.

The browser reviews the exact simulated envelope before signing: source, sequence, time bounds, pool call and arguments, authorization, simulation data, and fees. A deposit allows exactly its source-account authorization and the reviewed token transfer into the pool; transfers and withdrawals reject extra authorization entries. The review window is five minutes. An expired deposit review can be rebuilt for another explicit confirmation; refreshing the review does not submit it automatically.

The current private-action resource-fee cap is 20,000,000 stroops (2 XLM), separate from the reviewed classic inclusion fee. A simulated or assembled resource fee above that cap is rejected before signing; the wallet does not silently raise it. Restoration charges can make an otherwise small payment exceed the cap. A pre-signing fee-limit failure is not a ledger result, and a spend proof already shared during preparation still retains its input holds.

13.2 Held-balance self-recovery

An eligible held payment can be recovered by explicitly authorizing a new direct self-transfer of exactly its one or two reserved inputs to a fresh address owned by the same wallet. The current flow requires one pending action, no build reservation, and no active chained approval. It prepares a new proof against a currently usable anchor and separately asks for proof-sharing and signing consent. An atomic encrypted-state update replaces the pending attempt without making the inputs available in between.

This recovery control currently constructs a self-transfer, so it still requires free commitment slots. It does not offer a full-input public-exit option for reserved notes, and ordinary withdrawal selection excludes held inputs. Thus §15's contract-level saturation result does not make this particular wallet recovery control capacity-independent. A public withdrawal of held inputs would need a separate reviewed recovery path; the wallet never silently substitutes it.

This is a competing spend, not cancellation or instant unlocking. The replacement does not revoke the original proof: either can still confirm while its anchor is accepted and its real-input nullifiers remain unspent. Reconciliation distinguishes recovery confirmation, original-payment confirmation, another conflicting spend, and still pending. If neither spend confirms, the balance can remain held; the wallet never reports recovery merely because the replacement was approved, broadcast, or timed out.

13.3 Current-format state

StellarKey 1.0.0 is the starting application baseline. There are no supported pre-baseline deployments, record migrations or compatibility recovery routes. Encrypted pending actions require a direct route, explicit proof exposure and an outgoing-history policy. Signed actions also require their exact envelope expiry. Local build reservations are explicitly marked local; proof disclosure requires a durable pending action first. Issued-address history and the account's outgoing-history preference are explicit fields, including in a fresh wallet.

Unsupported records and encrypted backups fail validation without rewriting or deleting their bytes. The application does not sign, rebroadcast, look up transaction hashes or run recovery for an unsupported record. Current-format canonical reconciliation and held-balance self-recovery retain the conservative exposed-proof policy described above.

14 Local state and execution boundary

Owned notes, decrypted activity, outgoing metadata, canonical scan progress and checkpoints, and pending actions are encrypted and authenticated in deployment-bound IndexedDB records. A full encrypted wallet backup includes these sensitive records but excludes the disposable public Merkle cache.

Private panels and the proving worker load only after explicit intent. Lock, account/network change, leadership loss, manifest rejection, or a foreground safety failure clears in-memory private state and terminates the worker. A transient quiet-background failure retains the last verified snapshot and worker for a later retry. RPC-authentication disagreement retains the last authenticated snapshot in memory but disables actions and reports status-unknown. One scoped browser tab holds the synchronization lease; followers receive only redacted state and may take over after lease expiry.

Worker readiness is distinct from wallet/session authority. A failed worker can be recreated without reporting a wallet switch or discarding an already submitted action's durable tracking. Add funds waits for in-flight sync before preparation; worker recovery does not automatically retry a payment. Preparation and signing check wallet/session ownership across asynchronous steps. Revoking that ownership does not undo an already authorized durable commit or canonical submission tracking. Detached outcome watchers also capture revocable wallet/runtime authority, check it across polling and reconciliation awaits, and serialize journal classification with canonical synchronization. Revocation stops their later publication and polling without undoing an already authorized durable journal commit. Worker failure remains distinct from that authority. The September 12 follow-up in the action-context review records the focused coverage; this is not a claim that every asynchronous callback has been hardened.

Receive readiness is likewise separate from scanning: an existing scoped local address remains available during a read-only scan, while a missing address is not itself evidence of loading. Locked, other-tab, stopped/error, and retry states remain explicit. New-address publication and asynchronous QR/retry feedback are bound to the current wallet session.

The authenticated static catalogue supplies the deployment's checkpointed asset list without an RPC request during wallet unlock or the first switch to a Private tab. A live, two-provider registry read occurs only after the user selects **Refresh asset registry**, after an administrator changes the registry, or during an explicit runtime retry. This is how another device discovers a newly admitted asset or status change. A failed explicit refresh retains the last verified catalogue for the same wallet scope; it never adopts an uncorroborated or conflicting registry view.

JavaScript cannot guarantee physical memory erasure. The design assumes the served application, browser, operating system, device, dependencies, artifacts, and reviewed transaction are not compromised. A malicious extension or origin can defeat every browser-level privacy control.

15 Privacy boundary

Hidden for an internal transfer	Public or observable
Private transfer amount and asset	Pool contract and action timing
Recipient private address	Settlement flows: deposit and withdrawal token movements
Which note lanes are real	Anchor root, two nullifiers and three commitments
Whether a real output is recipient or change	Three fixed-size encrypted output packages
Memo plaintext	User's public submitting account and Stellar fee
Spending and viewing secrets	Transaction timing and network metadata

Deposits reveal their public source, asset, and amount. Withdrawals reveal their public recipient, asset, and amount. Internal transfers publish neither the asset contract nor registry index, and all registered assets share one action set. Protocol V2.1 publishes no action kind. Ordinary exact-input and change-producing wallet withdrawals share the same append behavior. Capacity exits remain distinguishable through their public append flag and unchanged tree; removing a kind label does not hide that exception. Fixed two-input/three-output arity, randomized lanes, and one matched clear diversifier per action obscure output roles, but they cannot create a large privacy set when the pool is small or activity is uniquely timed. Reusing a private receive address still links whole actions through the clear diversifier.

The wallet offers local-only amount and timing advice in deposit and withdrawal forms and reviews. It suggests considering rounder deposits with more than two decimal places, and warns when a withdrawal matches a same-asset deposit in available local history or follows a known same-asset deposit within 24 hours. Missing timestamps do not count as recent activity. These heuristics neither measure the anonymity set nor guarantee unlinkability; silence, waiting, or splitting a withdrawal is not evidence of privacy. They do not upload history, query additional RPC data, change amounts, or delay submission.

The RPC operator can see the connecting IP address, request timing, selected pool, queried ledger ranges, simulations, restoration attempts, and submitted transactions. With correctly authenticated HTTPS and uncompromised endpoints, a passive network observer sees connection endpoints, timing, sizes, and volume, but cannot directly read encrypted RPC request or response contents. This is the application-data confidentiality boundary provided by TLS. It does not prevent traffic analysis or correlation with the public ledger, and it does not conceal those requests from the RPC operator. Timing and pool activity remain public. Pool size, deposits, withdrawals, repeated public endpoints, private-address reuse, voluntary disclosure, or browser compromise may correlate otherwise hidden transfers.

Private Balance must not be described as hiding all identities, defeating all tracing, or guaranteeing privacy.

The separate Horizon stealth subsystem is complementary, not an alternative name for the pool. It derives a fresh classic destination for each payment to a reusable meta-address without proving, but the asset, amount, sender, timing, account creation, and later sweep can remain linkable. The two systems retain separate keys, recovery, sync, and user-facing guarantees.

15.1 Conditional security arguments

The following propositions concern the stated transition relation under the assumptions in §1. They are arguments about invariants, not a machine-checked proof of the complete implementation or a reduction for the entire deployed system. Finite tests can refute an invariant; they do not establish it for all possible executions.

Proposition 1 (note-value conservation). An accepted proof preserves the private-note accounting equation in §6 over integers, for one bound asset. The circuit range-checks individual values and the equal total below 2^{63}; the bounded sums cannot wrap around the BN254 scalar field. Real input membership and owner/nullifier relations bind consumed value, and the output commitments bind produced value. Deposit and withdrawal public values bind to the asset and endpoints reconstructed by the contract. Under knowledge soundness and commitment binding, an accepted action therefore cannot create extra spendable note value. This proposition assumes notes are counted by distinct positions and unspent nullifiers. It does not establish the underlying token's custody balance or undo a clawback.

Proposition 2 (cross-mode replay exclusion). A real note cannot be consumed twice by a normal spend and a FullInputExit in any order. Both derive its nullifier from the same context, nullifier key, note randomness, commitment and exact global leaf position; the action kind is not a nullifier domain separator. The contract rejects a nullifier already in its persistent spent set and records both spend nullifiers atomically. Concurrent valid invocations are serialized by the ledger. A failed invocation rolls back, and an archived spent marker remains spent on restoration. This argument assumes persistent-state correctness and nullifier collision resistance; it is not based on a client remembering a spend.

Proposition 3 (commitment saturation does not block full-input exit). Let a holder know a valid owned unspent note and a path to the available current root, with an admissible value, a valid proof, an executable contract, sufficient ledger resources and fees, and an available exact-transfer token balance. Then setting the public value to the sum of the selected one or two notes and all output values to zero yields the FullInputExit relation independently of n. The contract skips the append guard and leaves T, n unchanged. Its remaining checks are identical at $n = 3^{64}$ and at a smaller n. Pausing deposits or marking the asset ExitOnly does not remove this entrypoint. The claim excludes exhausted archive indices, unavailable proofs/state, compromised setup, token failure and partial withdrawals requiring change. It is an availability implication under explicit preconditions, not an unconditional liveness theorem.

Proposition 4 (authenticated replay consistency). Starting with an authenticated checkpoint, a client that verifies each next action index, current leaf position, canonical record encoding, chained digest, resulting tree and final authenticated contract head cannot accept a reordered, omitted or altered prefix without breaking hash binding or its head-authentication assumption. No-append exits advance the transcript and action cursor while preserving the leaf cursor; checking only the tree root would miss such records. Checkpoint and cache publication is atomic and overlap-checked. Corroborated RPC state remains a trust assumption, so this is not a light-client proof of Stellar consensus.

Confidentiality boundary. Groth16 zero knowledge concerns the private witness given the twelve public signals. It does not itself imply transaction-level unlinkability. The canonical action hash commits to public ciphertext bytes and endpoints; hiding amounts or recipients further depends on randomized note commitments, HPKE and client behavior. Public source accounts, common clear diversifiers, public settlement legs, boundary amounts, timing and selected asset candidates are part of the leakage model. We do not prove a compositional anonymity game for the browser, network, user behavior and ledger together. HPKE base mode is not sender authentication and does not give forward secrecy after recipient-key compromise. Ciphertext decryptability is outside the circuit, so a malicious sender can destroy its own recoverability without violating note conservation.

16 Evaluation and reproducibility

The evaluation asks three separate questions: whether the relation and state transitions enforce the intended invariants, whether the browser and contract agree on exact encodings, and what the integrated artifacts cost. A passing check answers its tested question only. Unless explicitly described as public Testnet readback, the workloads below are synthetic and contain no user funds.

Metric	Integrated V2.1 artifact
Constraints	40,647, compiled with Circom 2.2.3 `--O2`
Public / private inputs	12 / 410
R1CS	19,356,640 bytes
Witness Wasm	261,959 bytes
Development proving key	26,342,728 bytes
Point-compressed proving-key transport	17,747,434 bytes
Compressed transport size recorded in manifest	7,372,402 bytes
Canonical BN254 proof	256 bytes

Metric	Integrated V2.1 artifact
Pool Wasm, deployed unoptimized build	71,877 bytes
Verifier-only host test budget	29,623,260 instructions for deposit/transfer; 29,624,068 for withdrawal/exit

The last row, remeasured on September 22, 2026, measures the verifier crate under Soroban SDK 27.0.6 and Rust 1.97.1, with an unlimited test budget reset before each vector. Four real Groth16 vectors verify, covering inflow, quiet transfer, appending outflow, and non-appending outflow. These are native host-test instructions, not complete Wasm pool simulations, transaction fees or ledger throughput. The entrypoint, archive writes, token calls and dynamic ledger resource pricing add costs. V2.1 has twelve public signals. Verifier host cost does not establish browser proving performance or complete transaction resource use.

16.1 Validation method

The integrated circuit passes 14 tests, including actual proof generation and balanced negative exit witnesses. For each output lane, a negative witness reduces public withdrawal by one atomic unit and commits that unit as private change. Ordinary withdrawal accepts the balanced witness; FullInputExit rejects it. This distinguishes the zero-output exit restriction from conservation alone. Eight mutation tests deliberately remove security constraints and require their corresponding adversarial witnesses to stop being rejected. One mutation removes the exit's zero-output binding. Circomspect 0.9.0 and pinned CIVER commit `af7d4ed0325e6f7743d8a1ac0e415d0c69b8aae8` both detect their negative control. The September 22, 2026 V2.1 run in an isolated Linux container, with the clean pinned source checkout verified before execution, reports 21 verified components and 190,043 verified analysis constraints, zero failures and zero timeouts. The analyzer count is not the optimized R1CS count, and modular weak safety is not a complete protocol proof.

The full Rust workspace passes. Separately retained controlled Stellar Core archival results exercise the storage pattern described below; those results are not a fresh V2.1 deployment test. The archive probe exercises the persistent `has`, reject and set pattern at low TTL, restoration as spent and rejection of a duplicate after restoration. It is a test-only contract mirroring the storage pattern; it is not a full pool payment. The actual pool saturation test seeds a valid proof root and a synthetic full frontier, checks a high archive index, verifies exit, checks token-failure rollback and verifies shared normal-spend/exit replay protection. It does not populate the full tree or claim a reachable high-index Testnet history. Separate tree tests check the final carry and bounds, so the conditional saturation argument in §15 does not rely on that seeded state alone.

The browser protocol tests cover the primitives and recovery paths, including 1,000 Rust/TypeScript key/address differential cases. Exact-position tests cover boundaries beyond 32, 53, 64 and 100 bits, u128 archive encodings, full capacity, and unchanged-root exit checkpoint advancement. Wallet regression tests exercise durable holds, canonical confirmation and multi-note exit planning. The complete application, browser, accessibility and clean-release results must be recorded separately before a release; protocol checks do not replace them.

The current scanner processes one record at a time and runs at most three concurrent output tasks per record, despite a mapper cap of eight. Candidate assets for hidden-asset transfers are tried sequentially within each output task. Cold recovery is linear in retained archive records, with additional work for candidate assets and commitment hashing; a 64-level tree does not make

archive recovery constant cost. Warm caches validate checkpoints and overlap before atomic append. Normal spend paths require bounded depth-64 reads rather than a full-history reconstruction.

16.2 Integrated browser proving experiment

A loopback-only harness proves the integrated V2 circuit with a synthetic cross-subtree witness. It uses fresh browser contexts, fetches artifacts once per configuration, then records three sequential full-prove and verification samples. The host is Apple M3 Max, macOS ARM64; other local verification jobs were running, so these samples are descriptive smoke measurements, not controlled comparative performance estimates. There is no physical-phone claim.

Browser configuration	Full-prove samples, milliseconds
chromium default workers (151.0.7922.34)	2,069; 1,927; 1,949
webkit iphone emulation default workers (26.5)	3,137; 2,893; 2,959
chromium single thread (151.0.7922.34)	14,955; 14,650; 13,804
webkit iphone emulation single thread (26.5)	14,215; 13,859; 12,836

All twelve proofs verified. Artifact fetch time is excluded from full-prove time; first-use prover initialization is included in the first sample. Default workers and explicit single-thread mode differ substantially. Three samples do not support tail-latency or confidence-interval claims. Chromium exposed a sampled JavaScript heap counter; WebKit did not. Neither is a total process/Wasm peak-memory measurement. The reproducible harness and exact samples are run-capacity-browser.mjs and capacity-browser-v2.json.

16.3 Contemporary comparison

Only a benchmark that identifies the integrated artifact hashes is evidence for this baseline. No statistical performance comparison with another deployed shielded system has been conducted. No physical-phone result is claimed; iPhone or iPad emulation is not physical-device evidence.

Groth16 [1] gives small pairing-based proofs at the cost of circuit-specific setup. Zcash Orchard [5] instead uses Halo 2 over the Pallas/Vesta cycle and removes the trusted-setup requirement. Its specified construction is friendly to recursion, but ZIP 224 does not itself use recursive proofs. StellarKey's BLS12-381 option is not selected, and recursive proofs are not implemented. Changing either would require a reviewed construction, new artifacts and new deployments, not an editorial change to the security claim.

Penumbra's multi-asset shielded pool and tiered commitment tree [6] demonstrate that private notes, nullifiers and a shared hidden-asset tree are established patterns. Its three-tier quaternary tree uses a finite 48-bit position space and supports pruning irrelevant client subtrees. StellarKey's private 17+47 split is a fixed global ternary path; it currently retains a public authenticated node cache and does not implement Penumbra's pruned witness representation. RAILGUN's reference commitment code [7] retains roots by public tree number and rolls over depth-16 trees. Its public source-tree identifier and capacity model differ from the single hidden hierarchical witness here. These observations compare mechanisms, not measured anonymity or throughput across systems.

The defensible assessment is a development systems implementation using established techniques with explicit tradeoffs. BN254's reduced pairing-security margin [4, 8], single-party Phase 2 setup, visible transaction source and common clear diversifier prevent an unqualified state-of-the-art security or privacy claim. A stronger ranking would require a matched threat model, comparable workloads and devices, quantified leakage and independent review.

16.4 Artifact identity and reproduction

The following SHA-256 values identify the measured integrated artifacts:

Artifact	SHA-256
R1CS	`b59eaddeb2ec6eac403d2c2adee693ea2446abb094edcdd6f309456e1d77ae2f`
Witness Wasm	`185a6d8c684b9a1d4fbe878bf19ea1906afaf3f1137376b90235890b7ac128ed`
Development zkey	`6082d74328d861e5a2247083ec9e83f642b61fa89aef8b297f235d43ec1f07f0`
Verification-key JSON	`960b480dd4a10580e9e55e284eabdae072a359623b7530dd3bf40a399d64accf`
Pool Wasm	`88547981e9fe10861ba0e152a2185efd94ce5679e81822949bc9e39eb71ac39c`

Use locked dependencies and the pinned toolchains. The reproducibility entrypoint is `npm run private:check-reproducible` on the documented macOS ARM64 canonical build host. `npm run private:gate-a` runs static, weak-safety, zkey, circuit and mutation checks. `npm run test:private-protocol` runs browser-language protocol checks. Run `cargo test --workspace --locked` from `protocol/private-balance` for Rust checks. Two fresh isolated builds on September 13 produced identical R1CS, witness Wasm and pool Wasm hashes matching the table above. The implementation revision identifies the circuit, contract, runtime and validation tools; subsequent manuscript commits record documentation updates.

17 Deployment and trust status

This is a Live Testnet development deployment. The authenticated deployment catalogue advertises one live XLM/USDC development pool on Testnet:

`CBDJJPNO7SP7ZC3KXXSVL3B2V5B6WBX2WFRH4KJF2AMLX66KQ2LU33OD`.

XLM is registry index 0 and USDC index 1. The September 19 pool was deployed from a fresh synthetic Testnet account, with deployment checkpoint ledger 4,749,176 and post-registry ledger 4,749,178. Configuration, registered assets, unpaused deposits, empty archive and empty tree were read back after the deployment and registrations completed. The ephemeral deployment key was removed. Notes, addresses, proofs and encrypted state are accepted only in this deployment's current format and context. The retired September 13 protocol V2 pool `CAYCV26VCDNUEM6HHKQYHKDJ3O43CK5DCBT3TMMHFVEXE7CIFVWRY4R7` remains on-chain but is not in the authenticated catalogue, and current clients refuse it.

The deployment fixture records the locally selected Wasm hash and canonical ledger checkpoints. It does not contain a deployment transaction hash; a local file hash alone does not establish the

on-chain executable. A separate public read on September 13 at 01:01:01 UTC corroborated the instance executable through the SDF and Ankr RPCs at ledger 4,647,534. Both returned the Pool Wasm hash in §16; see capacity-v2-executable.json. This read establishes an observed executable match, not a locally verified consensus proof.

A separate isolated XLM lifecycle on September 19 confirmed a deposit at ledger 4,750,820, a self-transfer at 4,750,823, and a full-input exit at 4,750,826. All three used the deployed Wasm and locally verified Groth16 proofs. After each confirmation, the real archive reader and scanner rebuilt the canonical transcript from genesis. The exit advanced the action index by one and left the root and leaf count unchanged; the final full-history scan found no unspent test note. Keys and proof inputs remained in the test process; this was a command-line synthetic test, not a browser-wallet or USDC test, and it did not saturate the live tree. It explicitly exercised the contract exit branch before saturation; it does not validate the current wallet append policy.

The pre-submission minimum resource-fee estimates were 13,623,445, 13,045,043 and 13,292,204 stroops respectively. Each was below the application's 20,000,000-stroop cap. These are dated Testnet estimates excluding the separate classic inclusion fee, not realized net charges, fixed tariffs or throughput measurements. Evidence and the bounded runner are capacity-testnet-lifecycle.json and run-capacity-testnet.mjs. The runner requires `--testnet-synthetic` and starts with fresh test keys.

The dated XLM lifecycle and integrated browser experiment cover only their stated scenarios. They do not establish a complete browser-wallet or USDC lifecycle. Updating this paper does not renew measurements or constitute a security audit. Full application verification and human/device signoff remain separate release evidence.

The current proving key was created by a single-party setup. It passes `snarkjs zkey verify` against the pinned Powers-of-Tau transcript. The published PSE power-17 transcript [9], 151,088,274 bytes with SHA-256 `f807e065fde53f72f4bf4d57140fab85b26daa6cc95bdfec7cce93622b3a367c`, also passed full `snarkjs powersoftau verify` locally on September 13. This verifies the provided transcript's algebraic consistency; it does not attest participants' secret erasure or supply a public Phase 2 ceremony. The zkey check does not make the single-party development setup safe for real value. Retained Phase 2 trapdoor material permits forged proofs.

The application is deliberately Testnet-only and Mainnet rejects this feature. A Testnet reset deletes the pool and requires a redeploy, new evidence and new manifest hashes. Real-value promotion additionally needs independent circuit and contract review, a reviewed multi-party Phase 2 ceremony and transcript, reproducible builds, broader Wasm resource/fee evaluation, funded multi-asset lifecycle and recovery drills, and physical-device and human assistive-technology checks. The isolated XLM lifecycle above addresses only its stated scenario. Commitment saturation is addressed by the full-input exit relation, while archive growth, finite counters, partial-change capacity, underlying asset solvency/authorization and network availability remain explicit limitations.

18 Normative sources and operational guidance

This whitepaper explains the implemented design. Consensus and byte-level rules remain normative in the repository sources listed below.

Repository paths identify the normative source files at the implementation revision. Current artifact hashes identify the compiled V2.1 artifacts. Source links target the planned v1.1.0 release tag and become immutable references only after that tag is published. This manuscript currently

describes the working-tree candidate; the version marker alone does not establish a clean release or publication. Circuit and contract behavior determine what the deployment permits.

- Protocol V2.1 specification
- Threat model
- Canonical encoding
- Action circuit, witness construction, and authenticated deployment manifest
- Pool entrypoints and storage/TTL behavior
- Receive-address initialization and encrypted issuance/recovery state
- Recipient and archive scanner
- Public getter validation and archive-record restoration
- Action preparation, envelope review, and fee policy
- Held-proof recovery rules
- Wallet/runtime authority and outcome watchers and submission journal and polling
- Recovery guide
- Safe support guide
- Incident-response playbook

If this whitepaper conflicts with the generated manifest, conformance vectors, circuit, or contract compiled for a deployment, the deployment-bound code and authenticated artifacts govern that deployment and the documentation must be corrected.

References

[1] Jens Groth. *On the Size of Pairing-based Non-interactive Arguments.* EUROCRYPT 2016. IACR ePrint 2016/260. Paper.

[2] Lorenzo Grassi, Dmitry Khovratovich and Markus Schofnegger. *Poseidon2: A Faster Version of the Poseidon Hash Function.* IACR ePrint 2023/323, 2023. Paper.

[3] Richard Barnes, Karthikeyan Bhargavan, Benjamin Lipp and Christopher A. Wood. *Hybrid Public Key Encryption.* RFC 9180, February 2022. RFC.

[4] Stellar protocol contributors. *CAP-0074: BN254 Curve Support.* Final protocol specification, consulted September 13, 2026. Specification.

[5] Ying Tong Lai; maintained by Daira-Emma Hopwood, Jack Grigg, Sean Bowe and Kris Nuttycombe. *ZIP 224: Orchard Shielded Protocol.* Zcash protocol specification, consulted September 13, 2026. ZIP.

[6] Penumbra contributors. *Tiered Commitment Tree.* The Penumbra Protocol, consulted September 13, 2026. Specification.

[7] RAILGUN contributors. *Commitments.sol.* Reference implementation, commit `2afd011854d af35aad155a91d6bf2397e39ca90d`. Source.

[8] Michael Scott. *Pairing Implementation Revisited.* IACR ePrint 2019/077, 2019. Paper.

[9] Privacy and Scaling Explorations. *Perpetual Powers of Tau.* Published ceremony transcript catalogue, consulted September 13, 2026. Catalogue.

[10] Stellar Development Foundation. *Stellar Asset Contract* and *Fees, Resource Limits, and Metering.* Developer specifications, consulted September 13, 2026. Asset contract; resource model.